

Assignment III - FlexGen LLM Offloading and Memory Hierarchy Observation

Student ID: 314551147

1 Hardware Information

Path Used: Local Path

Item	Information
GPU	NVIDIA GeForce RTX 4070
CPU	13th Gen Intel(R) Core(TM) i5-13600K
RAM	16 GB
Disk	Local storage under WSL, 1007 GB total, 892 GB available
Operating System	WSL / Linux
CUDA Available	True

Table 1: Hardware and environment information

2 Overview

This report presents the experimental results and analysis of Assignment III. In this assignment, we use FlexGen to observe the memory hierarchy behavior of large language model inference.

FlexGen is an inference framework that can distribute a large language model’s weights, KV cache, and activations across GPU memory, CPU RAM, and disk. In this assignment, only the weight distribution is adjusted, while the KV cache and activations are kept entirely on GPU.

The model used in all experiments is **facebook/opt-1.3b**. The main goal is to observe how different weight placement strategies affect throughput, GPU memory usage, disk I/O behavior, and overall bottlenecks.

3 Environment Verification

Before running the full experiments, I first ran the 100% GPU baseline to verify that the environment was correctly set up.

Environment Verification Command

```
python -m flexllmgen.flex_opt \  
--model facebook/opt-1.3b \  

```

```
--percent 100 0 100 0 100 0 \  
--gpu-batch-size 4 \  
--prompt-len 128 \  
--gen-len 16 \  
--offload-dir flexgen_offload \  
2>&1 | tee logs/q1_gpu100.log
```

Result:

Verification Output

```
TorchDevice: cuda:0  
peak gpu mem: 2.806 GB  
prefill latency: 0.178 s  
prefill throughput: 2881.711 token/s  
decode latency: 0.261 s  
decode throughput: 229.574 token/s  
total latency: 0.439 s  
total throughput: 145.777 token/s
```

Explanation:

The program ran successfully without error messages. The CUDA device was correctly detected as `cuda:0`, and the model was successfully loaded and executed. Therefore, the environment setup was successful.

4 Question

Q1: Progressive Offload Sweep

Requirement:

Run five different weight distribution configurations with fixed prompt length, generation length, and GPU batch size.

- `--prompt-len 128`
- `--gen-len 16`
- `--gpu-batch-size 4`

Only the weight distribution is changed. The KV cache and activations are fixed at 100% GPU.

Commands

Q1-1: 100% GPU

```
python -m flexllmgen.flex_opt \  
--model facebook/opt-1.3b \  
--percent 100 0 100 0 100 0 \  
--gpu-batch-size 4 \  
--prompt-len 128 \  

```

```
--gen-len 16 \  
--offload-dir flexgen_offload \  
2>&1 | tee logs/q1_1_gpu100.log
```

Q1-2: 50% GPU + 50% CPU

```
python -m flexllmgen.flex_opt \  
--model facebook/opt-1.3b \  
--percent 50 50 100 0 100 0 \  
--gpu-batch-size 4 \  
--prompt-len 128 \  
--gen-len 16 \  
--offload-dir flexgen_offload \  
2>&1 | tee logs/q1_2_gpu50_cpu50.log
```

Q1-3: 50% GPU + 50% Disk

```
python -m flexllmgen.flex_opt \  
--model facebook/opt-1.3b \  
--percent 50 0 100 0 100 0 \  
--gpu-batch-size 4 \  
--prompt-len 128 \  
--gen-len 16 \  
--offload-dir flexgen_offload \  
2>&1 | tee logs/q1_3_gpu50_disk50.log
```

Q1-4: 50% CPU + 50% Disk

```
python -m flexllmgen.flex_opt \  
--model facebook/opt-1.3b \  
--percent 0 50 100 0 100 0 \  
--gpu-batch-size 4 \  
--prompt-len 128 \  
--gen-len 16 \  
--offload-dir flexgen_offload \  
2>&1 | tee logs/q1_4_cpu50_disk50.log
```

Q1-5: 100% Disk

```
python -m flexllmgen.flex_opt \  
--model facebook/opt-1.3b \  
--percent 0 0 100 0 100 0 \  
--gpu-batch-size 4 \  
--prompt-len 128 \  
--gen-len 16 \  
--offload-dir flexgen_offload \  
2>&1 | tee logs/q1_5_disk100.log
```

Result

Configuration	Weight Placement	Total Throughput	Peak GPU Memory
Q1-1	100% GPU	145.777 token/s	2.806 GB
Q1-2	50% GPU + 50% CPU	42.332 token/s	1.681 GB
Q1-3	50% GPU + 50% Disk	11.359 token/s	1.712 GB
Q1-4	50% CPU + 50% Disk	10.668 token/s	0.587 GB
Q1-5	100% Disk	6.224 token/s	0.556 GB

Table 2: Q1 throughput and peak GPU memory

Boundary Ratio and Bottleneck Analysis

Boundary	Ratio	Dominant Bottleneck (PCIe / Disk)
(1)→(2)	(1) / (2) = 3.44	PCIe
(2)→(3)	(2) / (3) = 3.73	Disk
(3)→(4)	(3) / (4) = 1.06	PCIe
(4)→(5)	(4) / (5) = 1.71	Disk

Table 3: Boundary ratio and dominant bottleneck analysis

Disk Type: Local storage under WSL on my machine.

Estimated Disk Bandwidth Order of Magnitude: Around GB/s order for local SSD storage. In comparison, PCIe transfer bandwidth is usually higher than disk bandwidth, while GPU memory bandwidth is much higher than both.

Analysis

(a) Within-class comparison:

For the PCIe boundaries, the ratio of (1)→(2) is 3.44, while the ratio of (3)→(4) is 1.06. These two PCIe boundary ratios are not close. The front-pair ratio is much larger than the back-pair ratio.

The reason is that (1)→(2) changes the system from fully GPU-resident weights to a configuration where half of the weights are stored in CPU RAM. This introduces CPU-to-GPU transfer through PCIe, so the slowdown is large. However, in (3)→(4), both configurations already include 50% disk offloading. Since disk I/O is already a major bottleneck, changing the other 50% from GPU to CPU adds only a small extra slowdown.

For the Disk boundaries, the ratio of (2)→(3) is 3.73, while the ratio of (4)→(5) is 1.71. These two Disk boundary ratios are also not close. The front-pair ratio is larger than the back-pair ratio.

This is because (2)→(3) changes half of the weights from CPU RAM to disk, so disk I/O is newly introduced into the weight loading path. In contrast, (4)→(5) already has 50% of the weights on disk. Moving the remaining 50% from CPU RAM to disk still causes slowdown, but the relative slowdown is smaller because the system is already disk-limited.

(b) Cross-class within-pair comparison:

In the front pair, the PCIe boundary (1)→(2) has a ratio of 3.44, while the Disk boundary (2)→(3) has a ratio of 3.73. The Disk ratio is slightly larger than the PCIe ratio. This means that introducing disk offloading causes a larger slowdown than introducing CPU RAM offloading.

In the back pair, the PCIe boundary (3)→(4) has a ratio of 1.06, while the Disk boundary (4)→(5) has a ratio of 1.71. The Disk ratio is again larger than the PCIe ratio. This shows that even when disk offloading already exists, moving more weights to disk still causes a larger slowdown than moving weights between GPU and CPU RAM.

Overall, Disk is the stronger bottleneck compared with PCIe. The reason is the bandwidth order of magnitude. GPU memory has the highest bandwidth, PCIe transfer between CPU RAM and GPU is slower, and disk storage is usually much slower than both. Therefore, when more weights are placed on disk, FlexGen spends more time waiting for data movement, which reduces the total throughput.

Q2: Batch Size and I/O Amortization

Requirement:

Use 100% disk offloading and compare different GPU batch sizes.

- `--percent 0 0 100 0 100 0`
- `--prompt-len 128`
- `--gen-len 16`
- GPU batch sizes: 1, 4, and 16

Commands

Q2-1: Batch Size 1

```
python -m flexllmgen.flex_opt \  
  --model facebook/opt-1.3b \  
  --percent 0 0 100 0 100 0 \  
  --gpu-batch-size 1 \  
  --prompt-len 128 \  
  --gen-len 16 \  
  --offload-dir flexgen_offload \  
  2>&1 | tee logs/q2_batch1.log
```

Q2-2: Batch Size 4

```
python -m flexllmgen.flex_opt \  
  --model facebook/opt-1.3b \  
  --percent 0 0 100 0 100 0 \  
  --gpu-batch-size 4 \  
  --prompt-len 128 \  
  --gen-len 16 \  
  2>&1 | tee logs/q2_batch4.log
```

```
--offload-dir flexgen_offload \
2>&1 | tee logs/q2_batch4.log
```

Q2-3: Batch Size 16

```
python -m flexllmgen.flex_opt \
--model facebook/opt-1.3b \
--percent 0 0 100 0 100 0 \
--gpu-batch-size 16 \
--prompt-len 128 \
--gen-len 16 \
--offload-dir flexgen_offload \
2>&1 | tee logs/q2_batch16.log
```

Result

Batch Size	Wall Time (Total Latency)	Total Throughput	Peak GPU Memory
1	8.907 s	1.796 token/s	0.457 GB
4	8.935 s	7.163 token/s	0.587 GB
16	9.028 s	28.355 token/s	1.019 GB

Table 4: Q2 batch size comparison

Answer:

When the GPU batch size increases from 1 to 4 and then to 16, the total throughput increases significantly. The throughput increases from 1.796 token/s to 7.163 token/s and then to 28.355 token/s. However, the total latency stays almost the same, changing only from 8.907 s to 9.028 s.

The throughput ratio from batch size 1 to batch size 4 is:

$$\frac{7.163}{1.796} \approx 3.99$$

The throughput ratio from batch size 4 to batch size 16 is:

$$\frac{28.355}{7.163} \approx 3.96$$

Both ratios are very close to the theoretical $4\times$ improvement because the batch size increases by 4 times in both comparisons.

Explanation:

In this experiment, all model weights are stored on disk. Therefore, FlexGen needs to load weights from disk during inference. The disk I/O cost is large, but this cost can be shared by more requests when the batch size becomes larger.

For batch size 1, the disk loading cost is used for only one request, so the throughput is low. For batch size 4 and 16, the same weight loading process can serve more requests in one batch. As a result, the total latency does not increase much, but the total throughput increases greatly.

This explains why the measured throughput ratios are close to the theoretical 4× scaling. The total wall time remains almost unchanged, while the number of requests processed in one batch increases by 4 times.

However, batch size cannot be scaled indefinitely. The peak GPU memory increases from 0.457 GB to 0.587 GB and then to 1.019 GB. This is because a larger batch size requires more GPU memory for KV cache, activations, and temporary buffers.

Therefore, increasing batch size can amortize disk I/O overhead and improve throughput, but it is eventually limited by available GPU memory.

Q3: Weight Compression and Tier Interaction

Requirement:

Compare the performance with and without `--compress-weight` in two settings:

1. 100% GPU weights
2. 100% Disk weights

Commands

Q3-1: 100% GPU without Compression

```
python -m flexllmgen.flex_opt \  
--model facebook/opt-1.3b \  
--percent 100 0 100 0 100 0 \  
--gpu-batch-size 4 \  
--prompt-len 128 \  
--gen-len 16 \  
--offload-dir flexgen_offload \  
2>&1 | tee logs/q3_gpu_no_compress.log
```

Q3-2: 100% GPU with Compression

```
python -m flexllmgen.flex_opt \  
--model facebook/opt-1.3b \  
--percent 100 0 100 0 100 0 \  
--gpu-batch-size 4 \  
--prompt-len 128 \  
--gen-len 16 \  
--compress-weight \  
--offload-dir flexgen_offload \  
2>&1 | tee logs/q3_gpu_compress.log
```

Q3-3: 100% Disk without Compression

```
python -m flexllmgen.flex_opt \  
--model facebook/opt-1.3b \  
--percent 0 0 100 0 100 0 \  
--gpu-batch-size 4 \  
--prompt-len 128 \  
2>&1 | tee logs/q3_disk_no_compress.log
```

```
--gen-len 16 \
--offload-dir flexgen_offload \
2>&1 | tee logs/q3_disk_no_compress.log
```

Q3-4: 100% Disk with Compression

```
python -m flexllmgen.flex_opt \
--model facebook/opt-1.3b \
--percent 0 0 100 0 100 0 \
--gpu-batch-size 4 \
--prompt-len 128 \
--gen-len 16 \
--compress-weight \
--offload-dir flexgen_offload \
2>&1 | tee logs/q3_disk_compress.log
```

Result

Configuration	Compression	Total Latency	Total Throughput	Peak GPU Memory
100% GPU	No	0.389 s	164.334 token/s	2.806 GB
100% GPU	Yes	0.883 s	72.450 token/s	1.103 GB
100% Disk	No	9.669 s	6.619 token/s	0.587 GB
100% Disk	Yes	9.184 s	6.969 token/s	0.475 GB

Table 5: Q3 weight compression comparison

Answer:

Weight compression reduces peak GPU memory in both the 100% GPU and 100% Disk configurations. For the 100% GPU configuration, peak GPU memory decreases from 2.806 GB to 1.103 GB. For the 100% Disk configuration, peak GPU memory decreases from 0.587 GB to 0.475 GB.

However, the effect on throughput is different in the two configurations. In the 100% GPU case, total throughput decreases from 164.334 token/s to 72.450 token/s after enabling weight compression. This is a slowdown of:

$$\frac{164.334}{72.450} \approx 2.27$$

Therefore, compression makes the 100% GPU case about $2.27\times$ slower.

In the 100% Disk case, total throughput slightly increases from 6.619 token/s to 6.969 token/s after enabling weight compression. This is a speedup of:

$$\frac{6.969}{6.619} \approx 1.05$$

Therefore, compression gives only about $1.05\times$ speedup in the disk offload case.

Explanation:

Weight compression reduces the amount of data needed to store and transfer model weights. This is why the peak GPU memory becomes lower when `--compress-weight` is enabled.

In the 100% GPU configuration, all weights are already stored in GPU memory. Since GPU memory bandwidth is very high, the original uncompressed configuration is already fast. After enabling compression, FlexGen needs extra decompression or dequantization work before computation. This additional overhead makes the compressed version slower, even though it uses less GPU memory.

In the 100% Disk configuration, the weights are stored on disk. Disk I/O is much slower than GPU memory access, so reducing the weight size can reduce the amount of data read from disk. Therefore, compression slightly improves throughput in the disk-offloading case. However, the improvement is small because compression also introduces decompression or dequantization overhead.

The throughput change is not close to the theoretical $4\times$ compression ratio because compression only reduces the weight size. It does not remove other costs such as GPU computation, cache handling, data movement, and decompression or dequantization overhead.

Therefore, weight compression is useful for reducing memory usage, but it does not always improve speed. It helps more when the bottleneck is disk I/O, but it can hurt performance when the weights are already fully stored on GPU.

Q4: I/O Behavior and Bottleneck Analysis

Requirement:

Analyze the disk I/O behavior and compare the timing breakdown between 100% GPU and 100% disk offloading.

Q4-1: Disk I/O Monitoring

Command:

Open two terminals.
In terminal A, run:

Terminal A: Monitor I/O

```
./monitor.sh logs/q4_1_io
```

In terminal B, run:

Terminal B: 100% Disk Offload

```
python -m flexllmgen.flex_opt \  
  --model facebook/opt-1.3b \  
  --percent 0 0 100 0 100 0 \  
  --gpu-batch-size 4 \  
  --prompt-len 128 \  
  --gen-len 16 \  
  --offload-dir flexgen_offload \  
  2>&1 | tee logs/q4_1_run.log
```

After terminal B finishes, I waited about 5–10 seconds and then stopped the monitor in terminal A with Ctrl+C.

Extract Peak Values from iostat Log

```
awk '$1=="sdd" {
    if ($2+0 > r) r=$2+0
    if ($3+0 > k) k=$3+0
    if ($7+0 > sz) sz=$7+0
    if ($NF+0 > u) u=$NF+0
} END {
    printf "r/s peak = %s\n", r
    printf "rkB/s peak = %s\n", k
    printf "rareq-sz peak = %.2f\n", sz
    printf "%util peak = %s\n", u
}' logs/q4_1_io_iostat.log
```

Result

iostat (disk offload)	Value
r/s peak	79
rkB/s peak	4500 KB/s
rareq-sz peak	100.03 KB
%util peak	42%

Table 6: Q4-1 disk I/O monitoring result

Answer:

The disk holding the weight files is **sdd**. The peak read request rate is 79 r/s, and the peak read bandwidth is 4500 KB/s. The peak average read request size is 100.03 KB, and the peak disk utilization is 42%.

Based on the **rareq-sz** value, the disk access pattern is closer to sequential large-block reading. This is because the average read request size is much larger than 32 KB.

Explanation:

The measured **rareq-sz** peak is 100.03 KB, which is larger than the 32 KB threshold. This indicates that FlexGen reads model weights from disk in relatively large chunks. Therefore, the I/O pattern is more similar to sequential large-block reads rather than random small-block reads.

The peak read bandwidth is 4500 KB/s, which is not very high for local storage. This may be because the experiment was performed under WSL and some model data may have been served from the operating system page cache. However, based on the request size, the observed read pattern is still closer to large-block reading.

Q4-2: Timing Breakdown

Commands:

Q4-2 Baseline: 100% GPU with Breakdown

```
python -m flexllmgen.flex_opt \
  --model facebook/opt-1.3b \
  --percent 100 0 100 0 100 0 \
  --gpu-batch-size 4 \
  --prompt-len 128 \
  --gen-len 16 \
  --debug-mode breakdown \
  --offload-dir flexgen_offload \
  2>&1 | tee logs/q4_2_baseline.log
```

Q4-2 Disk: 100% Disk with Breakdown

```
python -m flexllmgen.flex_opt \
  --model facebook/opt-1.3b \
  --percent 0 0 100 0 100 0 \
  --gpu-batch-size 4 \
  --prompt-len 128 \
  --gen-len 16 \
  --debug-mode breakdown \
  --offload-dir flexgen_offload \
  2>&1 | tee logs/q4_2_disk.log
```

Result

Scenario	load_weight	compute_layer_decoding	load / compute ratio
(1) Baseline	0.000021 s	0.001185 s	0.018
(5) Disk offload	0.011761 s	0.001216 s	9.67

Table 7: Q4-2 stage timing comparison

Answer:

The timing breakdown shows that the main difference between the 100% GPU baseline and the 100% Disk offload setting is the `load_weight` time. For the 100% GPU baseline, `load_weight` is only 0.000021 s per layer. For the 100% Disk offload setting, `load_weight` increases to 0.011761 s per layer.

The two `load_weight` values differ by:

$$\frac{0.011761}{0.000021} \approx 560$$

This means that the disk offload case has about 560× larger `load_weight` time, which is about 2.7 orders of magnitude.

However, the `compute_layer_decoding` time is almost the same in both settings. It is 0.001185 s per batch for the 100% GPU baseline and 0.001216 s per batch for the 100% Disk offload setting.

Explanation:

The `compute_layer_decoding` time represents the GPU computation time for each decoding layer. Since the model architecture, batch size, prompt length, and generation length are the same in both configurations, the GPU compute volume is almost unchanged. This shows that offloading does not significantly change the amount of GPU computation. It mainly changes where the model weights are stored and how they are moved to the GPU.

The `load_weight` time represents the overhead of loading model weights before computation. In the 100% GPU baseline, the weights are already stored in GPU memory, so loading weights is almost free. In the 100% Disk offload setting, the weights must be fetched from disk before computation, which greatly increases the `load_weight` time.

Therefore, the slowdown in the 100% Disk offload setting is mainly caused by data movement and disk I/O overhead, not by slower GPU computation. Offloading mainly changes the memory hierarchy and weight loading cost, rather than the GPU compute cost.

5 Conclusion

In this assignment, I used FlexGen to observe how different weight placement strategies affect LLM inference performance. The results show that placing all weights on GPU provides the best throughput but requires the most GPU memory. When weights are offloaded to CPU RAM or disk, throughput decreases because additional data transfer is required.

CPU offloading introduces PCIe transfer overhead, while disk offloading introduces much larger I/O overhead. Batch size can help amortize the I/O cost, but it also increases GPU memory usage. Weight compression can reduce the amount of data transferred, especially in disk-bound settings, but it also introduces decompression overhead.

Overall, this assignment demonstrates that memory hierarchy and data movement are important performance factors in large language model inference.

6 Submission Files

The submitted zip file should follow the required format:

Submission Format

```
HW3_314551147.zip
HW3_314551147/
  314551147_report.pdf
  q4_1_io_iostat.log
  q4_1_io_free.log
  q4_2_baseline.log
  q4_2_disk.log
```